1. When first accessing the assignment, I got an ECDSA key message. An ECDSA key is a protocol key that is used to verify the identity of the remote server. This typically pops up the first time, as the computer verifies the server connection.

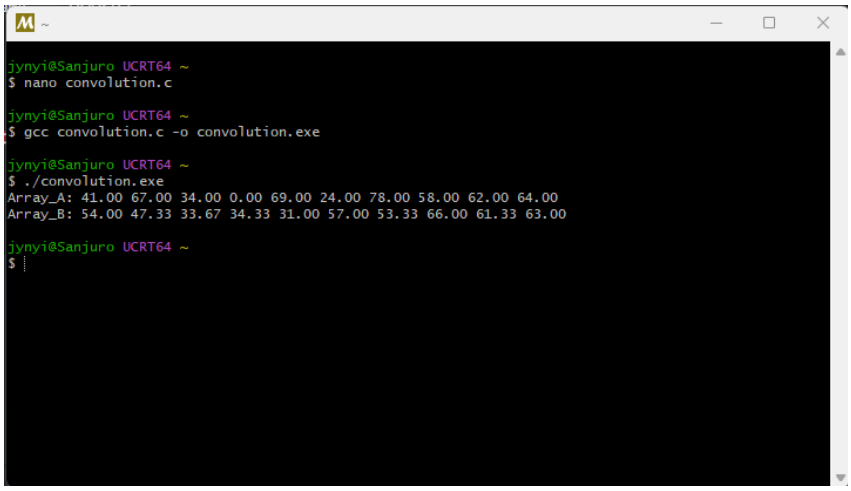
2. The commands -l and -l help to access more from the standard ls. -l displays more info about the files, and -a list all files, including hidden ones.

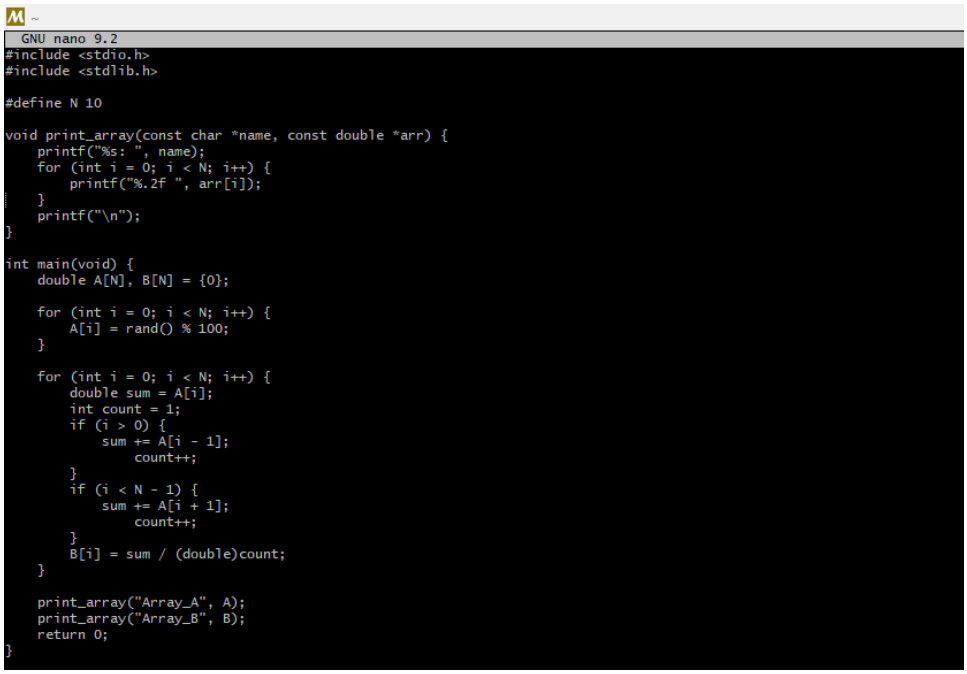
3. Hiding files and directories helps by reducing the work environment. This helps in multiple ways, from isolating certain configurations, sources, and code to preventing external modifications to files.

A = {[initial array values]} // Input array of fixed size N, initialized to random values
B = {0,0,0,...} // Output array of fixed size N, initialized to zeros

for i from 0 to N-1 do:
sum = A[i]
if i > 0 then:
sum = sum + A[i-1] // Add left neighbor if it exists
if i < N-1 then:
sum = sum + A[i+1] // Add right neighbor if it exists
B[i] = sum / 3 // Each element in B is just the average of the corresponding element in A and its neighbors

Print Array_A // Output the original array
Print Array_B // Output the convolved array





```
GNU nano 9.2
#include <stdio.h>
#include <stdlib.h>

#define N 10

void print_array(const char *name, const double *arr) {
    printf("%s: ", name);
    for (int i = 0; i < N; i++) {
        printf("%.2f ", arr[i]);
    }
    printf("\n");
}

int main(void) {
    double A[N], B[N] = {0};

    for (int i = 0; i < N; i++) {
        A[i] = rand() % 100;
    }

    for (int i = 0; i < N; i++) {
        double sum = A[i];
        int count = 1;
        if (i > 0) {
            sum += A[i - 1];
                count++;
        }
        if (i < N - 1) {
            sum += A[i + 1];
                count++;
        }
        B[i] = sum / (double)count;
    }

    print_array("Array_A", A);
    print_array("Array_B", B);
    return 0;
}
```

4. Following the data structures, I implemented the algorithm using 2 arrays. The first generates it, and the second contains the sorted array. Then, through the for loop, the neighbors of each index are summed, averaged, and stored in array B.

5. Comparing C to other programming languages, it is most similar to Java. What is similar is the import util being replaced with #include, the structure of stacks still contains the method(condition) { variables }, and the variable and logic syntax remaining the same.

6. The first and last elements are set to 0 due to out-of-bounds logic. To improve accuracy, I added a count to check whether an index has 1 neighbor or 2, instead of simply dividing by 3.

7. **C code to RISC-V:**

```
.LC0:
        .string "%s: "
.LC1:
        .string "%.2f "
print_array:
        addi    sp,sp,-48
        sw      ra,44(sp)
        sw      s0,40(sp)
        addi    s0,sp,48
        sw      a0,-36(s0)
        sw      a1,-40(s0)
        lw      a1,-36(s0)
        lla     a0,.LC0
        call    printf@plt
        sw      zero,-20(s0)
        j       .L2
.L3:
        lw      a5,-20(s0)
        slli    a5,a5,3
        lw      a4,-40(s0)
        add     a5,a4,a5
        fld     fa5,0(a5)
        fsd     fa5,-48(s0)
        lw      a2,-48(s0)
        lw      a3,-44(s0)
```

```
        lla     a0,.LC1
        call    printf@plt
        lw      a5,-20(s0)
        addi    a5,a5,1
        sw      a5,-20(s0)
.L2:
        lw      a4,-20(s0)
        li      a5,9
        ble     a4,a5,.L3
        li      a0,10
        call    putchar@plt
        nop
        lw      ra,44(sp)
        lw      s0,40(sp)
        addi    sp,sp,48
        jr      ra
.LC2:
        .string "Array_A"
.LC3:
        .string "Array_B"
main:
        addi    sp,sp,-208
        sw      ra,204(sp)
        sw      s0,200(sp)
        addi    s0,sp,208
        addi    a5,s0,-200
```

```
        li      a4,80
        mv      a2,a4
        li      a1,0
        mv      a0,a5
        call    memset@plt
        sw      zero,-20(s0)
        j       .L5
.L6:
        call    rand@plt
        mv      a4,a0
        li      a5,1374388224
        addi    a5,a5,1311
        mulh    a5,a4,a5
        srai    a3,a5,5
        srai    a5,a4,31
        sub     a5,a3,a5
        li      a3,100
        mul     a5,a5,a3
        sub     a5,a4,a5
        fcvt.d.w        fa5,a5
        lw      a4,-20(s0)
        addi    a5,s0,-120
        slli    a4,a4,3
        add     a5,a4,a5
        fsd     fa5,0(a5)
        lw      a5,-20(s0)
```

```
        addi    a5,a5,1
        sw      a5,-20(s0)
.L5:
        lw      a4,-20(s0)
        li      a5,9
        ble     a4,a5,.L6
        sw      zero,-24(s0)
        j       .L7
.L10:
        lw      a4,-24(s0)
        addi    a5,s0,-120
        slli    a4,a4,3
        add     a5,a4,a5
        fld     fa5,0(a5)
        fsd     fa5,-32(s0)
        li      a5,1
        sw      a5,-36(s0)
        lw      a5,-24(s0)
        ble     a5,zero,.L8
        lw      a5,-24(s0)
        addi    a4,a5,-1
        addi    a5,s0,-120
        slli    a4,a4,3
        add     a5,a4,a5
        fld     fa5,0(a5)
        fld     fa4,-32(s0)
```

```
        fadd.d  fa5,fa4,fa5
        fsd     fa5,-32(s0)
        lw      a5,-36(s0)
        addi    a5,a5,1
        sw      a5,-36(s0)
.L8:
        lw      a4,-24(s0)
        li      a5,8
        bgt     a4,a5,.L9
        lw      a5,-24(s0)
        addi    a4,a5,1
        addi    a5,s0,-120
        slli    a4,a4,3
        add     a5,a4,a5
        fld     fa5,0(a5)
        fld     fa4,-32(s0)
        fadd.d  fa5,fa4,fa5
        fsd     fa5,-32(s0)
        lw      a5,-36(s0)
        addi    a5,a5,1
        sw      a5,-36(s0)
.L9:
        lw      a5,-36(s0)
        fcvt.d.w        fa5,a5
        fld     fa4,-32(s0)
        fdiv.d  fa5,fa4,fa5
```

```
        lw      a4,-24(s0)
        addi    a5,s0,-200
        slli    a4,a4,3
        add     a5,a4,a5
        fsd     fa5,0(a5)
        lw      a5,-24(s0)
        addi    a5,a5,1
        sw      a5,-24(s0)
.L7:
        lw      a4,-24(s0)
        li      a5,9
        ble     a4,a5,.L10
        addi    a5,s0,-120
        mv      a1,a5
        lla     a0,.LC2
        call    print_array
        addi    a5,s0,-200
        mv      a1,a5
        lla     a0,.LC3
        call    print_array
        li      a5,0
        mv      a0,a5
        lw      ra,204(sp)
        lw      s0,200(sp)
        addi    sp,sp,208
        jr      ra
```

8. Yes, while some instructions are similar, overall the process is more complex than anything I have worked with before. Specifically fsd, fadd, fcvt are instructions I have never used before.

9. Compared to my solution, the compiled code follows the same arithmetic but differs in random value generation. C has libraries for math and uses rand(), but RISC-V must link to a library for rand generation.

10. While the arithmetic logic is the same, it solves it differently. In C, “% 100” is used to restrict the rand generation to 0 to 99. On RISC-V, that process is more complicated, requiring x - (x / 100) * 100, instead of having the % operand

11. I would say the C code is better, as overall it is neater and more efficient. Not only does it have standard libraries, but it is also a better compiler for more complex functions. This allows the data structures in C code to be more condensed yet just as efficient as RISC.